AUDITMOS

Secure your space

COOKIE3

audit / code review report

July 28, 2025

TABLE OF CONTENTS

1. License
2. Disclaimer
3. Approach and methodology
4. Description
5. Audit scope
6. Findings

contact@auditmos.com

AUDITMOS

Secure your space

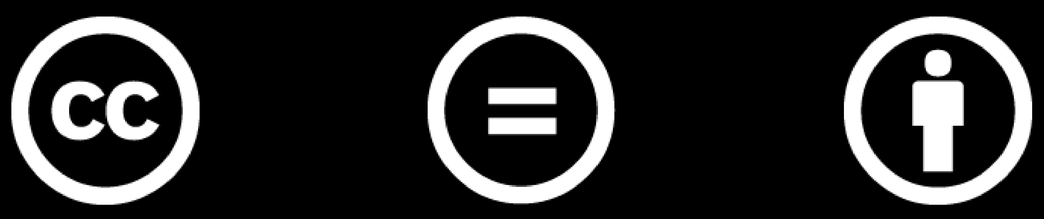
COOKIE3

audit / code review report

July 28, 2025

LICENSE

Attribution-NoDerivatives 4.0 International (CC BY-ND 4.0)



contact@auditmos.com

AUDITMOS

Secure your space

COOKIE3

audit / code review report

July 28, 2025

DISCLAIMER

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

contact@auditmos.com

AUDITMOS

Secure your space

COOKIE3

audit / code review report

July 28, 2025

APPROACH AND METHODOLOGY

PURPOSE

1. Determine the correct operation of the protocol, according to the design specification.
2. Identify possible vulnerabilities that could be exploited by an attacker.
3. Detect errors in the smart contract that could lead to unexpected behavior.
4. Analyze whether best practices were followed during development.
5. Make recommendations to improve security and code readability.

CODEBASE

GitHub	https://github.com/Cookie3-dev/community-gated-leaderboards
Branch	main
Contracts	*

METHODOLOGY

1. Reading the available documentation and understanding the code.
2. Doing automated code analysis and reviewing dependencies.
3. Checking manually source code line by line for security vulnerabilities.
4. Following guildlines and recommendations.
5. Preparing this report.

AUDITMOS

Secure your space

COOKIE3

audit / code review report

July 28, 2025

DESCRIPTION

Issues Categories:

Severity	Description
CRITICAL	vulnerability that can lead to loss of funds, failure to recover blocked funds, or catastrophic denial of service.
HIGH	vulnerability that can lead to incorrect contract state or unpredictable operation of the contract.
MEDIUM	failure to adhere to best practices, incorrect usage of primitives, without major impact on security.
LOW	recommendations or potential optimizations which can lead to better user experience or readability.

Each issue can be in the following state:

State	Description
PENDING	still waiting for resolving
ACKNOWLEDGED	know but not planned to resolve for some reasons
RESOLVED	fixed and deployed

contact@auditmos.com

AUDITMOS

Secure your space

COOKIE3

audit / code review report

July 28, 2025

AUDIT SCOPE

1. getting to know the project	✓
2. research into architecture	✓
3. manual code read	✓
4. permissions of state changing functions	✓
5. identify common Solidity vulnerabilities	✓
6. test coverage	✓
7. static analysis	✓
8. storage key overlaps	✓
9. DOS possibilities by malicious attacker	✓
10. steal funds possibilities	✓

contact@auditmos.com

AUDITMOS

Secure your space

COOKIE3

audit / code review report

July 28, 2025

FINDINGS

Finding	Severity	Status
#1 - getSubscriptions is vulnerable to DoS	LOW	RESOLVED
#2 - updateDiscount allows updates to non-existent discount IDs	LOW	RESOLVED
#3 - Use of .transfer() in _withdrawCoin may cause reverts	LOW	RESOLVED
#4 - Zero duration subscription abuse	LOW	RESOLVED
#5 - Assignment in event emission	LOW	RESOLVED
#6 - Rename _withdrawCoin to _withdrawNative for clarity	LOW	RESOLVED

contact@auditmos.com

AUDITMOS

Secure your space

COOKIE3

audit / code review report

July 28, 2025

#1 - GETSUBSCRIPTIONS IS VULNERABLE TO DOS

Severity	Status
LOW	RESOLVED

The getSubscriptions function returns a list of all registered subscriptions. It iterates through all project IDs stored in projectsIds and fills the return array by casting each subscription to a DTO using toDTO function.

```
/// @notice Returns the list of the subscriptions.
function getSubscriptions() external view returns (Subscription.DTO[] memory) {
    uint256[] memory ids = projectsIds.values();

    uint256 size = ids.length;
    Subscription.DTO[] memory collection = new Subscription.DTO[](size);
    for (uint256 i = 0; i < size; i++) {
        uint256 id = ids[i];
        collection[i] = subscriptions[id].toDTO(id);
    }

    return collection;
}
```

This implementation introduces a potential Denial of Service (DoS) vulnerability due to unbounded array growth. As the number of project IDs increases, the function may eventually exceed block gas limits, causing transactions that call it to fail. This would make the function unusable and potentially break any relying off-chain or on-chain functionality.

RECOMMENDATION

Introduce pagination by allowing the caller to specify a start index and a limit to retrieve subscriptions in batches:

```
function getSubscriptions(uint256 start, uint256 limit) external view returns (Subscription.DTO[] memory);
```

COOKIE3

audit / code review report

July 28, 2025

#2 - UPDATEDISCOUNT ALLOWS UPDATES TO NON-EXISTENT DISCOUNT IDS

Severity	Status
LOW	RESOLVED

The updateDiscount function allows a manager to update a discount's minimum and maximum duration, as well as its rate. While it correctly validates the discount attributes via Discount.validate, it does not check whether the discountId actually exists.

```
    function updateDiscount(uint256 discountId, Discount.Attributes memory
discount) external onlyRole(MANAGER_ROLE) {
        if (!Discount.validate(discount)) revert Errors.UnacceptableValue();

        discounts[discountId] = discount;

        emit Discount.Updated(discountId, discount);
    }
```

This omission introduces a logic flaw: the function can silently create a "ghost" discount a discountId that exists in the mapping discounts but not in the enumerable set that tracks registered discount IDs. This can lead to inconsistencies in how discounts are retrieved or listed.

RECOMMENDATION

Before updating the discount, verify that the discountId is already registered in the enumerable set:

```
if (!discountsIds.contains(discountId)) revert Errors.DiscountNotFound();
```

COOKIE3

audit / code review report

July 28, 2025

Severity	Status
LOW	RESOLVED

#3 - USE OF .TRANSFER() IN _WITHDRAWCOIN MAY CAUSE REVERTS

Using .transfer() to transfer native assets is compiled to a low-level CALL opcode with a gas limitation of 2300 during compilation. In case the recipient is a smart contract with a fallback function that consumes more than 2300, the call will fail, and the entire call to _withdrawCoin will revert.

```
/// @notice Withdraws the native coin to the recipient.
/// @param to Address of the recipient.
function _withdrawCoin(address payable to) internal virtual {
    if (to == address(0)) revert WithdrawToZeroAddress();
    to.transfer(address(this).balance);
}
```

RECOMMENDATION

Consider replacing .transfer() with .call() instead.

COOKIE3

audit / code review report

July 28, 2025

#4 - ZERO DURATION SUBSCRIPTION ABUSE

Severity	Status
LOW	RESOLVED

The subscribe() function allows registration of a project with a duration of 0 and an invalid (non-existent) discount ID.

RECOMMENDATION

Add validation inside subscribe():

```
if(!discountIds.contains(discountId)) revert Errors.DiscountNotFound();
```

contact@auditmos.com

COOKIE3

audit / code review report

July 28, 2025

#5 - ASSIGNMENT IN EVENT EMISSION

Severity	Status
LOW	RESOLVED

Avoid assigning new values directly within an event emission. It makes the code harder to read.

```
emit FeeUpdated(fee.rate = rate, fee.collector = collector);

// Same here:
return discount.calculate(duration * fee.rate, duration);
```

RECOMMENDATION

Separate the state mutation from the event emission for clarity and safety:

```
fee.rate = rate;
fee.collector = collector;

emit FeeUpdated(rate, collector);
```

This improves readability, avoids unintended side effects and aligns with best practices.

COOKIE3

audit / code review report

July 28, 2025

Severity	Status
LOW	RESOLVED

#6 - RENAME _WITHDRAWCOIN TO _WITHDRAWNATIVE FOR CLARITY

Rename the _withdrawCoin function to _withdrawNative for improved readability and clarity.

The term "coin" is ambiguous and may be confused with ERC20 tokens or project-specific terminology. Using "native" more clearly communicates that the function deals with the blockchain's native asset (e.g., ETH, AVAX, MATIC).

```
/// @notice Withdraws the native coin to the recipient.
/// @param to Address of the recipient.
function _withdrawCoin(address payable to) internal virtual {
    if (to == address(0)) revert WithdrawToZeroAddress();

    to.transfer(address(this).balance);
}
```

RECOMMENDATION

Rename _withdrawCoin to _withdrawNative.

auditmos.com

AUDITMOS

Secure your space

contact@auditmos.com